

GUIDE METHODOLOGIQUE

Comment construire les diagrammes UML

Cas d'utilisation • Sequence • Classes et objets • Packages

Synthese realisee a partir du support de cours
« Ingenierie du developpement logiciel » - J.-P. Babau, UBO

Document de revision - M1 Informatique / ILIA

Sommaire

1. Qu'est-ce qu'UML et à quoi ça sert
2. Le diagramme de cas d'utilisation
3. Le diagramme de séquence d'analyse
4. Le diagramme de classes et d'objets
5. Le diagramme de séquence de conception
6. Le diagramme de packages
7. Démarche générale : de l'idée au code
8. Récapitulatif de toutes les règles

1 Qu'est-ce qu'UML et a quoi ca sert

UML (*Unified Modeling Language*) est un **langage graphique de modelisation** : des boites, fleches, lignes et ellipses permettent de definir des **diagrammes** qui representent differentes vues d'un meme systeme.

Important : UML n'est pas un langage de programmation oriente objet. Un diagramme de classes UML n'est pas un diagramme de classes Java : UML sert a decire une vue de haut niveau de la structure et du comportement d'un systeme, pas a ecrire directement du code.

UML permet de :

- eliminer les ambiguïtes et les imprecisions des descriptions informelles ;
- faciliter l'analyse et les evolutions du systeme ;
- rendre les modeles productifs (generation de code, transformation de modeles) ;
- documenter la production logicielle, reflechir et communiquer a son propos.

Les 14 types de diagrammes UML

UML definit 14 diagrammes, repartis en deux grandes familles :

Aspect comportemental (dynamique)	Aspect structurel (statique)
Diagramme de cas d'utilisation	Diagramme de classes
Diagramme d'activites	Diagramme d'objets
Diagramme d'etats-transitions	Diagramme de composants
Diagrammes d'interaction : sequence, communication, global d'interaction, temps	Diagramme de deploiement
	Diagramme de packages
	Diagramme de structures composites

Dans une optique de developpeur, quatre diagrammes suffisent le plus souvent a couvrir tout le cycle (de l'expression du besoin au code) : le **diagramme de cas d'utilisation**, le **diagramme de sequence** (analyse puis conception), le **diagramme de classes / d'objets**, et le **diagramme de packages**. Ce guide se concentre sur ces quatre diagrammes et sur la methode pour les construire.

Demarche generale. On part du flou du cahier des charges pour aller vers le code :

1. **Cas d'utilisation** -> qu'est-ce que le systeme doit faire, vu par ses utilisateurs ?
2. **Sequence d'analyse** -> comment se deroule chaque scenario, quels messages s'echangeant acteurs et systeme ?
3. **Classes et objets** -> quelle structure statique (classes, attributs, relations) supporte ces echanges ?
4. **Sequence de conception** -> quels appels de methode entre objets realisent chaque scenario ?
5. **Packages** -> comment organiser les classes en modules coherents et peu couples ?

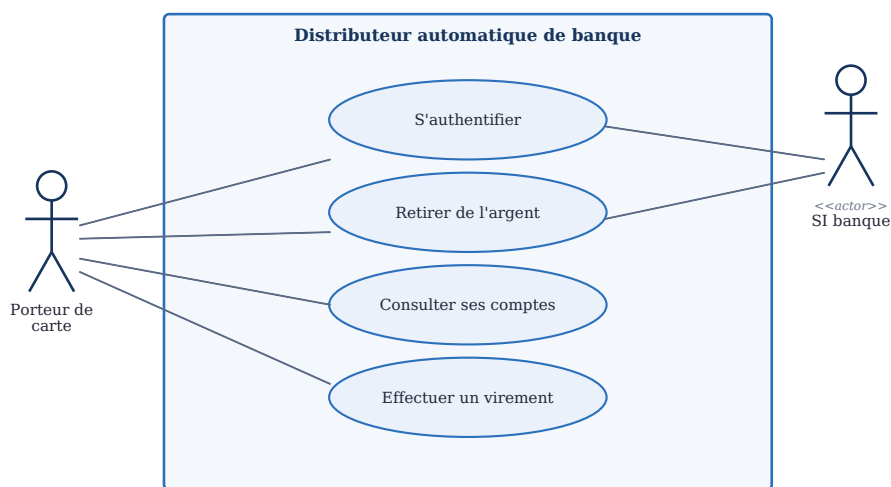
2 Le diagramme de cas d'utilisation

A quoi ca sert

Le diagramme de cas d'utilisation (*use case*) sert à **fixer les limites du système** et à modéliser le point de vue des utilisateurs : quelles sont les fonctionnalités du système, sans dire comment elles sont réalisées. C'est un premier pas pour passer du flou de l'énoncé à des exigences de fonctionnalités. Sa notation est volontairement simple et compréhensible par tous : c'est avant tout un **outil de communication**, pas un outil formel.

Les éléments de notation

Acteur	role (personne ou autre système) qui interagit avec le système ; dessiné sous forme de bonhomme filiforme, ou avec le stereotype <<actor>> quand l'acteur est lui-même un système.
Cas d'utilisation	une fonctionnalité du système, représentée par une ellipse contenant un verbe à l'infinitif.
Système	un rectangle qui délimite la frontière du logiciel modélisé ; le nom du système est indiqué en haut du rectangle.
Association	trait reliant un acteur à un cas d'utilisation auquel il participe.



Exemple : distributeur automatique de banque (acteur principal "Porteur de carte", acteur secondaire "SI banque")

Acteurs primaires et secondaires

Un cas d'utilisation a en général un **acteur principal** : celui qui initie le cas d'utilisation et pour lequel le système rend un service. Il peut aussi impliquer un ou plusieurs **acteurs secondaires**, sollicités par le système pour réaliser le cas d'utilisation (par exemple, le système bancaire central sollicite pour valider un code secret).

Méthode pas à pas

- 1 Delimiter le système : dessiner le rectangle et lui donner un nom.
- 2 Identifier les acteurs : quels rôles (personnes, autres systèmes) interagissent avec le système ? Un acteur existe indépendamment du système.
- 3 Identifier les cas d'utilisation : pour chaque acteur, terminer la phrase "Tiens, j'utiliserais bien ce système pour ...". Chaque réponse est un cas d'utilisation.
- 4 Relier chaque acteur aux cas d'utilisation auxquels il participe par un trait d'association.
- 5 Vérifier que chaque cas d'utilisation est réalisable par le système seul ou avec l'aide d'un acteur secondaire explicitement identifié.
- 6 Ne pas représenter les communications internes au système (bases de données, composants internes ne sont pas des acteurs).

Règles à respecter (CU)

Règle	Énoncé
-------	--------

CU1	Les acteurs existent independamment du systeme.
CU2	Un cas d'utilisation doit pouvoir completer la phrase "Tiens, j'utiliserais bien ce systeme pour ..."
CU3	Chaque cas d'utilisation est realisable soit par le systeme lui-meme, soit a l'aide d'un acteur secondaire.
CU4	Le rectangle systeme represente tout le logiciel : ses elements constitutifs (bases de donnees, etc.) ne sont pas des acteurs.
CU5	On ne represente pas les communications internes du systeme.
CU6	La description d'un acteur est independante de ses utilisations du systeme.
CU7	Dans un scenario associe, une action est numerotee.
CU8	Une action concerne soit un echange systeme/acteur, soit une operation interne.

Un diagramme de cas d'utilisation est en somme le **sommaire des fonctionnalites** du systeme : il resume l'ensemble des scenarios d'utilisation possibles, qui seront ensuite detailles un a un par des diagrammes de sequence.

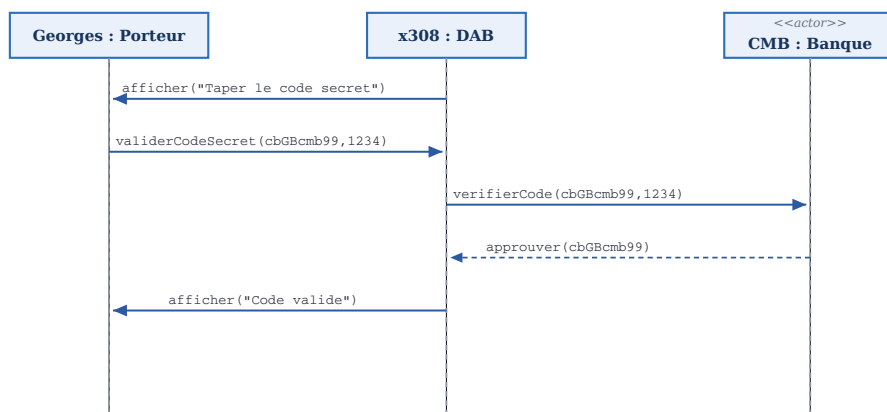
3 Le diagramme de sequence d'analyse

A quoi ca sert

Le diagramme de sequence d'analyse illustre **l'execution d'UN scenario** (une "user story") d'un cas d'utilisation : quels messages sont echanges entre les acteurs et le systeme, et dans quel ordre. Le temps s'ecoule de haut en bas, sans echelle de temps. C'est une base de tests pour la suite : si le programme ne se comporte pas comme le montre le diagramme, il y a un probleme.

Les elements de notation

- Entite / ligne de vie** instance d'un acteur ou l'application elle-meme ; nommee en haut, avec une ligne verticale pointillee representant l'ecoulement du temps.
- Message** fleche horizontale entre deux lignes de vie, portant le nom du message et ses parametres valuees (jamais generiques).
- Reference (ref)** renvoie vers un autre diagramme de sequence (un sous-scenario deja decrit ailleurs), pour eviter la duplication.



Scenario nominal du cas d'utilisation "S'authentifier"

Methode pas a pas

- 1 Choisir UN cas d'utilisation et UN scenario precis a l'interieur de celui-ci (nominal, ou une variante alternative / d'erreur).
- 2 Ecrire d'abord le scenario en langage naturel, sous forme d'une liste d'actions numerotees (une "user story").
- 3 Placer une ligne de vie par entite impliquee : les acteurs (nommes, ex. "Georges : Porteur de carte") et le systeme ou ses sous-parties.
- 4 Traduire chaque action de la user story en un message horizontal, avec des parametres concrets (valeurs reelles, pas des variables generiques).
- 5 Pour les scenarios alternatifs (erreurs, cas particuliers), faire un diagramme separe et y faire reference (Ref : . . .) depuis le scenario nominal si besoin.

Regles a respecter (DS)

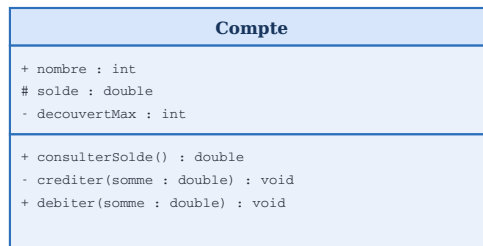
Regle	Enonce
DS1-2	Les entites sont des instances d'acteur ou l'application deployee ; elles ont un nom.
DS2	Un diagramme correspond a UNE execution particuliere (un seul scenario).
DS3	Un message possede des parametres values, non generiques.
DS4	Un diagramme de sequence d'analyse illustre l'execution d'un scenario avec les echanges entre acteurs et systeme, ainsi que les operations internes.

4 Le diagramme de classes et d'objets

A quoi ca sert

Le diagramme de sequence d'analyse est un premier pas vers l'implementation objet : les messages prefigurent des methodes, et leurs parametres prefigurent les parametres de ces methodes. Il manque encore les classes qui composeront le systeme : c'est l'objet du diagramme de classes, qui donne la **vue statique et structurelle** du systeme (classes, attributs, methodes, relations). Il a un lien direct avec les langages de programmation objet (Java, C++...) et sert a la fois en analyse et en conception : architecture du code, generation de code, documentation.

Notation d'une classe



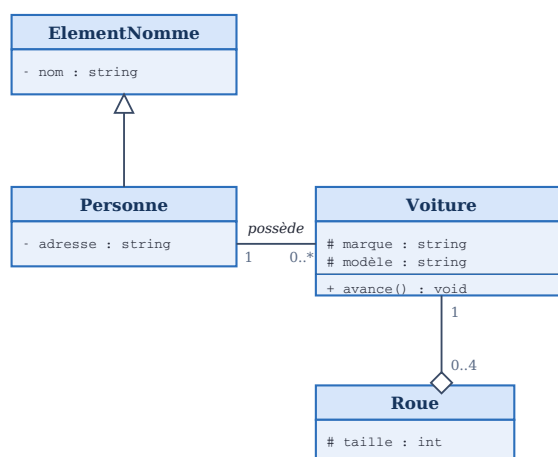
Notation d'une classe : nom, attributs, methodes

- + (public) accessible par tous
- # (protected) accessible uniquement par les classes heritieres
- (private) accessible uniquement a l'interieur de la classe
- souligne** attribut ou methode statique
- italique* classe ou methode abstraite

Les relations entre classes

L'association

Une association represente une relation entre deux classes. Elle est **toujours nommee** (souvent un verbe) et porte des **cardinalites** (multiplicites) de chaque cote : un nombre positif ou nul, ou *. On peut indiquer le sens de lecture avec une petite fleche pres du nom.



*Personne herite de ElementNomme ; une Personne possède 0 a * Voiture(s), une Voiture appartient a exactement une Personne ; une Voiture est composee de 0 a 4 Roue(s)*

La dependance

Relation unidirectionnelle : la modification de la classe cible peut impliquer une modification de la classe source.

Utilisee typiquement quand une classe utilise une autre comme type de parametre ou d'attribut (equivalent d'un import en Java). Notation : fleche en tirets, stereotype <<use>>.

L'agregation

Cas particulier d'association unidirectionnelle exprimant une relation de contenance, sans nom (le "contient" est implicite). Notation : losange vide du cote du contenant.

La composition

Agregation plus forte : le composant et les composees sont fortement lies, la creation du composant entraine celle des composees, et la destruction du compose entraine celle de tous ses composants. Pas de nom sur l'association. Notation : losange plein.

L'heritage (generalisation)

La classe fille possede toutes les caracteristiques de sa (ses) classe(s) parente(s), sauf les membres prives. Elle peut redefinir une methode (meme signature). Toutes les associations de la classe parente s'appliquent aux classes derivees. Notation : fleche a pointe triangulaire vide vers le parent.

L'interface et la realisation

Une interface regroupe des methodes abstraites assurant un service, tres proche de la notion Java. Une classe qui l'implemente est reliee par une fleche en tirets a pointe triangulaire, stereotype <<realize>>.

Methode pas a pas pour elaborer un diagramme de classes

- 1 Partir des diagrammes de sequence deja ecrits : les entites qui echantent des messages et les messages eux-memes suggerent des classes candidates et des methodes.
- 2 Identifier les classes "metier" (noms des entites recurrentes dans les scenarios).
- 3 Pour chaque classe, lister ses attributs (avec visibilite et type) et ses methodes (avec visibilite, parametres types, type de retour).
- 4 Relier les classes par des associations simples : donner un nom et les cardinalites.
- 5 Privilieger les associations directionnelles (un seul sens de navigation) pour reduire le couplage : c'est plus simple a implementer et cela reduit les dependances.
- 6 Identifier les relations de contenance forte (composition) et faible (agregation).
- 7 Identifier les hierarchies d'heritage : une sous-classe doit etre un cas particulier reel de la classe parente (pas seulement un moyen de reutiliser du code).
- 8 Extraire des interfaces pour separer l'usage d'un objet de sa mise en oeuvre, quand c'est pertinent.

Regles a respecter (DC / DA)

Regle	Enonce
DC1	Les noms de classes commencent par une majuscule.
DC2	Les noms d'attributs et de methodes commencent par une minuscule.
DC3	Tous les noms refletem clairement les concepts manipules.
DC4	Nommage selon la convention camelCase.
DC5	Un attribut a une visibilite et un type.
DC6	Un parametre de methode a un type.
DC7	Une methode a une visibilite et un type de retour.
DA1	Pour les diagrammes de conception, on privilegie les associations simples (directionnelles).
DA2	Si la classe A contient la classe B (composition) : A ne peut exister sans B, et vice-versa.
DA3	Un objet d'une sous-classe est un objet (specifique) de la classe parente.
DA4	Une association simple possede un nom et une ou deux cardinalites (un nombre positif ou nul, ou *).

Attention : un diagramme de classes UML n'est pas un diagramme de classes Java. UML donne une vue de haut niveau ; le comportement detaille (le corps des methodes) n'apparait pas dans ce diagramme, il est decrit par les diagrammes de sequence de conception.

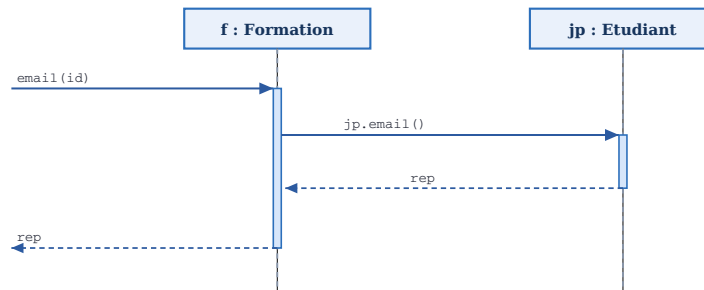
Diagramme d'objets

Un diagramme d'objets montre des **instances** concretes de classes a un instant donne (avec leurs valeurs d'attributs). Il est complementaire au diagramme de classes : utile quand la structure generale, avec ses cardinalites, est difficile a se représenter mentalement a partir du seul diagramme de classes.

5 Le diagramme de sequence de conception

Difference avec la sequence d'analyse

Une fois le diagramme de classes etabli, on peut detaillier les diagrammes de sequence : les entites deviennent des **objets** (instances des classes definies), et les messages deviennent des **appels de methode** entre ces objets. C'est toujours UNE trace d'execution du programme -- proche, dans l'esprit, d'une session de debogage.



Appel de methode entre objets : f (Formation) delegate a jp (Etudiant)

Methode pas a pas

- 1 Declarer chaque objet implique sous la forme nomObjet : NomClasse.
- 2 Représenter la creation d'objet quand elle a lieu (equivalent du new Java).
- 3 Ne montrer que les appels de methode reellement possibles : on n'envoie un message a un objet que s'il est accessible (attribut de la classe courante, ou variable locale declaree dans la methode).
- 4 Indiquer les parametres valeurs des appels, et les valeurs de retour des methodes.
- 5 Verifier la coherence avec le diagramme de classes : chaque appel doit correspondre a une association existante ou a une variable locale.

Regles a respecter (DS)

Regle	Enonce
DS1-2	Les entites sont des instances d'objets, declarees avec un nom d'objet suivi du nom de classe.
DS3	Un message possede des parametres values, non generiques.
DS5	Un diagramme de sequence de conception illustre les appels de methode entre les objets d'une application.
DS6	On n'envoie un message a un objet que si c'est un attribut de la classe ou une variable declaree dans la methode.

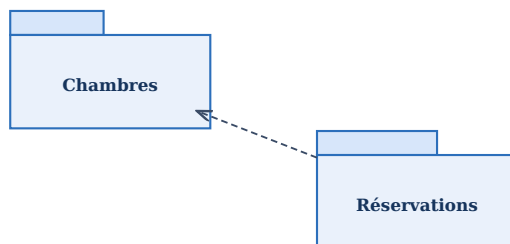
6 Le diagramme de packages

A quoi ca sert

Le decoupage d'un diagramme de classes en packages sert a le rendre plus facilement comprehensible d'un seul coup d'oeil, et a faciliter la reutilisation d'un ensemble de classes.

Deux concepts clés

- **Coherence** : un package regroupe des classes fortement liees d'un point de vue semantique -- fonctionnalites de meme nature, meme degre d'evolution (stable vs. amenee a changer), ou duree de vie des instances similaire.
- **Independance** : les classes de deux packages differents doivent etre (presque) independantes les unes des autres.



Un package "Reservations" depend du package "Chambres" (une Reservation ne va pas sans la Chambre concernee, l'inverse n'est pas vrai)

Methode pas a pas

- 1 Regrouper les classes du diagramme de classes en ensembles coherents semantiquement.
- 2 Pour chaque paire de packages en relation, identifier le sens de la dependance reelle (qui a besoin de qui pour exister).
- 3 Privilegier un seul sens de navigation entre packages, comme pour les associations entre classes.
- 4 Verifier qu'il n'existe pas de dependance croisee entre deux packages (A depend de B et B depend de A) : c'est un signe qu'il faut revoir le decoupage.

Regle a respecter (DP)

Regle	Enonce
DP1	Il n'y a pas de dependance croisee entre les packages.

7 Demarche generale : de l'idee au code

Le cours resume la demarche de conception en trois grandes questions, qui correspondent chacune a un ou plusieurs diagrammes :

- 1 **Quel va etre le comportement du systeme, sa dynamique ?** Identifier et detailler les principales fonctionnalites -> *diagramme de cas d'utilisation* (les "user stories"), puis *diagrammes de sequence d'analyse* (les traces d'execution de chaque scenario).
- 2 **Quelle va etre sa structure ?** Identifier les objets qui gerent ces fonctionnalites et les liens entre eux -> *diagramme de classes et d'objets*, complete par les *diagrammes de sequence de conception* (appels de methode), puis organise en *diagramme de packages*.
- 3 **On peut commencer a coder** en suivant le modele -- puis, en retour, on peut aussi modeliser a partir d'un programme existant pour le documenter ou verifier sa coherence avec le modele.

Un diagramme n'est pas un programme : la vue graphique (communication, structure) reste une abstraction de la technologie et du code. On modelise pour reflechir, communiquer et verifier -- pas pour remplacer le code.

8 Recapitulatif de toutes les regles

Diagramme	Regle	Enonce
Cas d'utilisation	CU1	Les acteurs existent independamment du systeme.
	CU2	Un cas d'utilisation complete "Tiens, j'utiliserais bien ce systeme pour ..."
	CU3	Realisable par le systeme seul, ou avec un acteur secondaire.
	CU4	Le rectangle systeme = tout le logiciel ; ses composants internes ne sont pas des acteurs.
	CU5	Pas de communications internes representees.
	CU6	Description de l'acteur independante de ses utilisations.
	CU7	Une action de scenario est numerotee.
	CU8	Une action = echange systeme/acteur ou operation interne.
Sequence (analyse)	DS1 - 2	Entites = instances d'acteur ou l'application, nommees.
	DS2	Un diagramme = UNE execution particuliere.
	DS3	Message avec parametres valuees, non generiques.
	DS4	Illustre l'execution d'un scenario (acteurs + operations internes).
Classes et objets	DC1	Classe : majuscule initiale.
	DC2	Attribut / methode : minuscule initiale.
	DC3	Noms clairs, refletant les concepts.
	DC4	Convention camelCase.
	DC5	Attribut : visibilite + type.
	DC6	Parametre de methode : type.
	DC7	Methode : visibilite + type de retour.
	DA1	Associations simples (directionnelles) privilegiees.
	DA2	Composition : A ne peut exister sans B et vice-versa.
	DA3	Sous-classe = objet specifique de la classe parente.
	DA4	Association simple : nom + une ou deux cardinalites.
Sequence (conception)	DS5	Illustre les appels de methode entre objets.
	DS6	Message envoye seulement si l'objet est accessible (attribut ou variable locale).
Packages	DP1	Pas de dependance croisee entre packages.

Document de synthese pedagogique. Voir le second document pour des exemples complets de diagrammes accompagnes de leur code Java equivalent.